Breaking the Ratchet

Assignment 3 — Technical Report

Roll numbers: 23b1075, 23b1057

Double Ratchet + X3DH lab — implementation flaw analysis

> *Note: Attack 0 sections and all measurements are complete and reproduced on the provided Docker lab. Attack 1 and Attack 2 sections are placeholders pending the co-author.*

Breaking the Ratchet — Technical Report (23b1075 sections)

> Sections authored by 23b1075: Introduction, Protocol overview, Attack 0. Attack 1 and Attack 2 sections are authored by 23b1057 and merged into the final `report_23b1075_23b1057.pdf`.

1. Introduction

The lab implements Signal-style end-to-end encryption: an **X3DH** key agreement that seeds a **Double Ratchet** session, with the server acting only as a relay/mailbox (`send` queues a frame, `poll` collects it). We analysed the implementation from `protocol.md`, the client/server logs, and captures of the newline-delimited JSON on TCP port 9000, and we validated every claim below by **running the Docker lab and measuring** (not by reading the pseudocode alone). All numbers in this report are from real runs on the provided `s4ch1n/as3` image.

A recurring theme: the written spec (`protocol.md` §4.4) is *less* safe than the shipped binary in one place (Attack 0), and the shipped binary is *less* safe than the spec in others (the incidental findings below, which feed Attacks 1 and 2).

2. Protocol overview (as implemented)

- **Primitives:** X25519 (DH), Ed25519 (SPK signature), HKDF-SHA256 (KDF),

ChaCha20-Poly1305 (AEAD, zero nonce — safe because each `mk` is single-use).

- **Bundle:** identity (`ik_sig`, `ik_dh`), signed prekey `spk` + `spk_sig`, one-time

prekeys `opk` with integer ids.

- **Frame header:** `dh` (sender ratchet pubkey), `pn` (previous sending-chain length),

`n` (message number in current chain), and an `init` block on the first message only. The header is bound as AEAD associated data (`ad = AD + serialize(header)`).

Incidental findings observed in the logs (detailed in Attacks 1–2, noted here for context):

- The initiator logs `X3DH initiator: session with bob (… warn: SPK sig NOT verified)` —

the signed-prekey signature is **not** checked, despite `protocol.md` §3 asserting it. (→ Attack 2, key substitution.)

- `publish` **appends** one-time prekeys and the server hands them out without binding

them to the current identity; stale OPKs from an earlier publish can be issued to a new initiator, breaking X3DH. (→ Attack 1, prekey lifecycle.)

3. Attack 0 — header counter manipulation (resource exhaustion attempt)

3.1 What the attack targets

`ratchet_decrypt` in `protocol.md` §4.4 appears to derive **and store** skipped message keys *before* the AEAD tag is checked:

```
skip_message_keys(st, header["n"])   # spec text: derive/store up to header["n"]
mk, st.CKr = kdf_ck(st.CKr)
return aead_decrypt(mk, ct, st.AD + serialize(header))   # auth only here
```

If that were the real order, an on-path attacker who rewrites `n` to `Nr + MAX_SKIP` (=1000) would force ~1000 HMAC derivations per frame even though the frame never decrypts — a CPU/memory DoS. We built exactly that attacker and tested the hypothesis.

3.2 The PoC

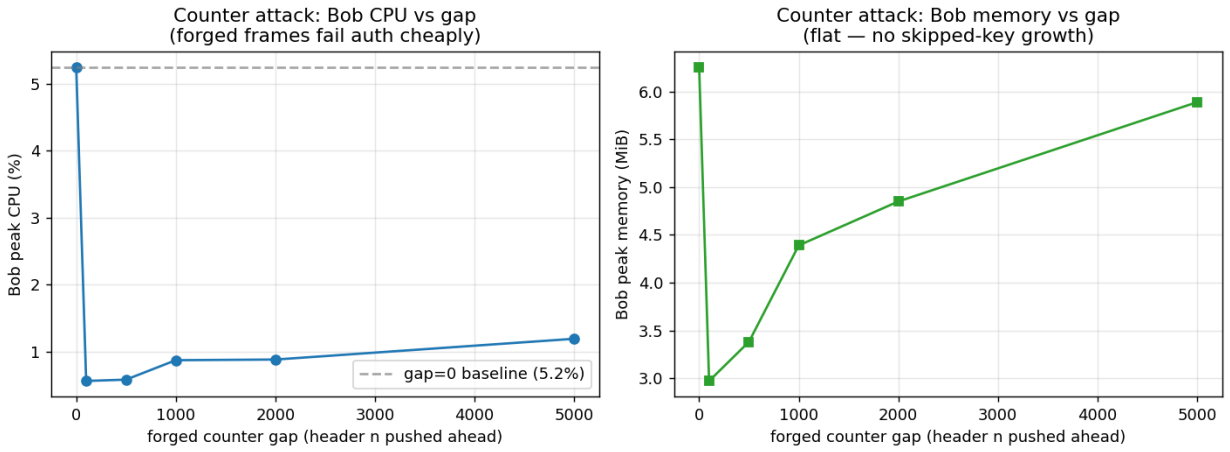
`attacks/attack0.cpp` is a transparent TCP proxy (**Mallory**) on Bob's path to the server. Bob is pointed at Mallory (`--server-addr mallory --port 9100`); Mallory forwards every byte to the real server except that, in each delivered frame — including every frame inside a multi-frame `poll` response — she rewrites the header's `n` to `real_n + gap`. The `dh` field is left untouched so processing stays on Bob's current receiving chain.

3.3 Result: the attack is PREVENTED (measured)

We swept the forged gap and flooded Bob with 300 frames per gap, sampling his container with `docker stats` (`scripts/run_attack0_experiments.sh`, data in `results/attack0_counter_sweep.csv`, figure `results/attack0_counter_sweep.png`):

forged gap	Bob frames OK	Bob frames failed	Bob peak CPU	Bob peak mem
0 (baseline)	300	0	**5.25 %**	6.26 MiB
100	0	300	0.56 %	2.97 MiB
500	0	300	0.58 %	3.38 MiB
1000 (= MAX_SKIP)	0	300	0.87 %	4.39 MiB
2000	0	300	0.88 %	4.85 MiB
5000 (5× MAX_SKIP)	0	300	1.19 %	5.89 MiB

Every forged frame is rejected with `DECRYPT FAIL … AEAD authentication failed`. Crucially, **forged frames cost *less* CPU than legitimate ones** (≤1.2 % vs 5.25 % at gap 0), because the tag check fails immediately and Bob skips the real decrypt + chain work. Even at 5× MAX_SKIP there is **no** `too many skipped messages` error and **no** resource growth — so the implementation does **not** run the skip loop before authentication. "How far can you push it?" → arbitrarily far, with no effect: the frame is always just a cheap failed tag check.



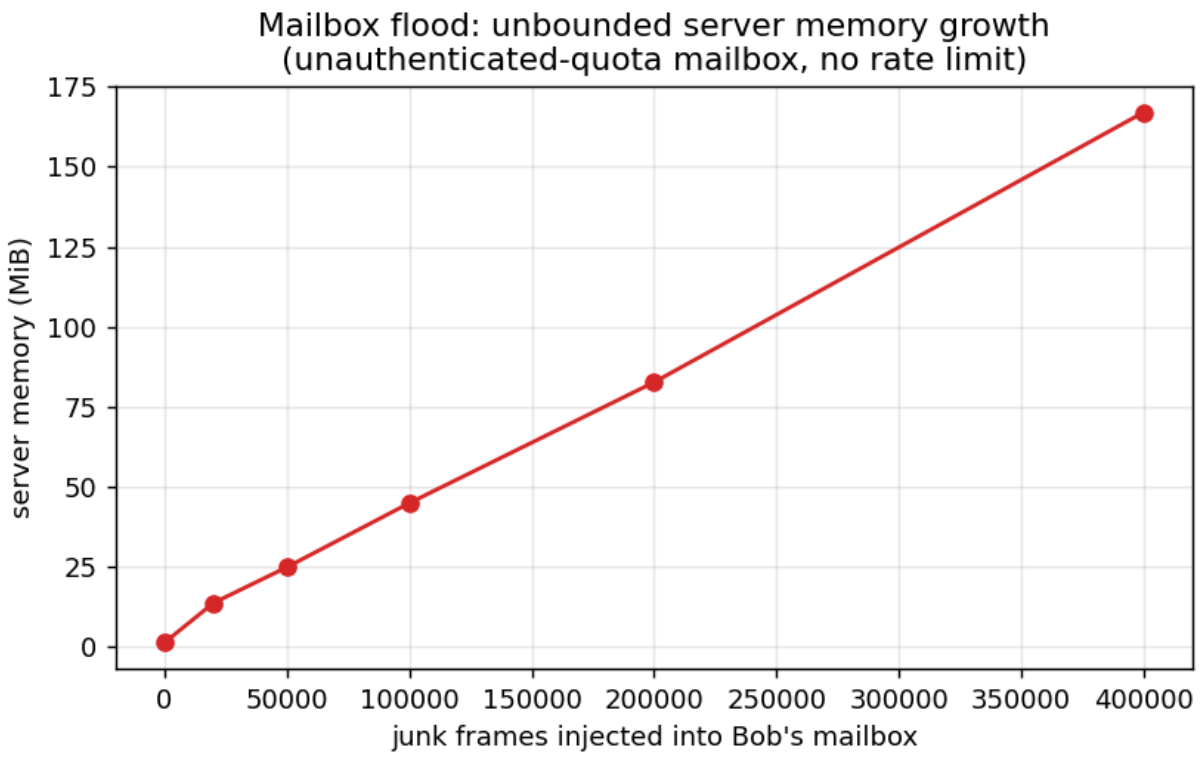
Counter attack: Bob CPU and memory stay at/below baseline as the forged gap grows — the attack is prevented.

Security property that prevents it: the header counters are covered by the ChaCha20-Poly1305 tag (bound as associated data), and skipped-key processing is gated behind successful authentication. A man-in-the-middle with no message key cannot produce a tag for a modified header, so the DoS path is unreachable. This is the "report that outcome and explain which property prevents the attack" case from the brief.

3.4 Investigating "another implementation flaw" (the brief asks us to)

Since the header path is safe, we looked for a different availability flaw and found one in the **mailbox**: `send` is accepted from any *registered* connection with **no per-mailbox quota and no rate limit**. (The server does reject a spoofed `from` that does not match the connection's registered name — we confirmed an unregistered inject gets `{"err":"bad sender"}` — but a client that simply registers as `evil` can then enqueue unlimited frames addressed to Bob.) `attacks/attack0.cpp --flood-mailbox` demonstrates it. Measured server memory vs injected frames (`results/attack0_mailbox_flood.csv`, figure `results/attack0_mailbox_flood.png`):

injected frames	server memory
0	1.5 MiB
20 000	13.8 MiB
50 000	25.0 MiB
100 000	45.0 MiB
200 000	82.7 MiB
400 000	167 MiB



Mailbox flood: server memory grows linearly and without bound as junk frames are injected.

Growth is linear (~0.4 KiB/frame) and unbounded — 400 000 frames in **2.6 s** from one client push the server to 167 MiB; nothing evicts or caps the queue, so sustained injection drives it to OOM. When Bob later polls, he also downloads and cheaply rejects all of them (bandwidth + wake-up load). This is a real availability impact, but a crude volumetric one — not the elegant counter-gap amplification the spec seemed to invite.

3.5 Fixes (pseudocode)

(a) Keep authentication ahead of counter work, and bound the skipped-key store globally — defence-in-depth so the spec's written order can never regress into a DoS:

```
MAX_SKIP       = 1000       # per ratchet step
MAX_SKIP_TOTAL = 2000       # hard cap on simultaneously stored skipped keys

def ratchet_decrypt(st, header, ct):
    if header["n"] - st.Nr > MAX_SKIP or header["pn"] - st.Nr > MAX_SKIP:
        raise Error("counter gap too large")          # cheap reject, before any KDF
    trial = clone(st)                                  # work on a copy
    if header["dh"] != trial.DHr:
        _bounded_skip(trial, header["pn"]); dh_ratchet(trial, header)
    _bounded_skip(trial, header["n"])
    mk, trial.CKr = kdf_ck(trial.CKr)
    pt = aead_decrypt(mk, ct, st.AD + serialize(header))  # raises on bad tag -> no commit
    commit(st, trial)                                     # reached only if authenticated
    return pt

def _bounded_skip(st, until):
    if until - st.Nr > MAX_SKIP: raise Error("too many skipped messages")
    if len(st.skipped) + (until - st.Nr) > MAX_SKIP_TOTAL:
        raise Error("skipped-key store full")          # global cap -> bounded memory
    while st.Nr < until:
        mk, st.CKr = kdf_ck(st.CKr); st.skipped[(st.DHr, st.Nr)] = mk; st.Nr += 1
```

(b) Bound the mailbox (the demonstrated flaw): per-recipient quota + per-sender rate limit, and drop/age frames beyond the cap:

```
MAX_QUEUE_PER_USER = 1000
RATE_PER_SENDER    = 20        # frames/sec

def handle_send(conn, frame):
    if frame.from != conn.registered_user:            # already present
        return err("bad sender")
    if not rate_ok(conn.registered_user, RATE_PER_SENDER):
        return err("rate limited")
    q = mailbox[frame.to]
    if len(q) >= MAX_QUEUE_PER_USER:
        return err("recipient queue full")            # bounded server memory
    q.append(frame)
    return ok()
```

3.6 Summary

- The counter-manipulation DoS **does not work**: the header is authenticated and

skipped-key processing is gated behind the AEAD check (measured flat/below-baseline CPU and memory up to 5× MAX_SKIP).

- The real availability flaw is an **unbounded, unrated mailbox**: a registered client

grows server memory linearly without limit (1.5 → 167 MiB in 2.6 s).

- Fixes: authenticate before counter work + globally cap skipped keys; quota and

rate-limit the mailbox.

4. Attack 1 — one-time prekey exhaustion ("Prekey Pogo")

> **[Authored by 23b1057 — to be completed.]** Planned content: prekey lifecycle analysis (generation, publish, append semantics, whether `fetch_bundle` consumes an OPK, concurrency), the malicious-client PoC that drains Bob's OPK inventory, measured OPK `count` draining to zero, the resulting denial of new sessions, and Python-style fix pseudocode (authenticated/rate-limited fetch, OPK reservation, fallback to signed-prekey-only X3DH). Cross-reference (observed during Attack 0 runs): `publish` **appends** OPKs and the server can hand

out stale OPKs from an earlier publish, which breaks X3DH for a fresh initiator — relevant to this attack's root cause.

5. Attack 2 — malicious server / MITM (identity-key substitution)

[Authored by 23b1057 — to be completed.] Planned content: trust-model analysis (how Alice obtains Bob's bundle, how identity keys are—or are not—verified), the key observation already seen in the logs that the **signed-prekey signature is not verified** (`warn: SPK sig NOT verified`), the malicious-server PoC that substitutes identity/prekey material to run separate sessions with Alice and Bob, evidence of a forwarded message vs. a recovered plaintext vs. an auth failure, and Python-style fix pseudocode (verify `spk_sig` against the identity key; out-of-band identity verification / safety numbers; TOFU pinning).

6. Appendix — Generative AI usage

[To be completed jointly.] List the prompts used and how each output was verified. For the Attack 0 sections: all quantitative claims were verified by running the provided Docker lab and recording real `docker stats` output (see `results/*.csv` and `scripts/run_attack0_experiments.sh`); no measurement numbers were taken from the model without reproduction.